CS216 Object-Oriented Programming: Java Programming Mastery Problem Set

Seventy realistic, integrated Java programming assignments aligned to the CS216 (R-24) syllabus, with the practical, requirement-driven style of the CS 253 Lab Cycle 1 and Lab Cycle 2 sheets.

Introduction and How to Use This Set

Every problem describes a believable software scenario, lists precise requirements, and states the constraints and business rules your program must respect. Each one asks you to combine several Java features in a single application. There are no solutions, hints, code snippets, or expected outputs. Design decisions such as class structure, method signatures, and data representation are yours to make unless a constraint fixes them.

Unless a problem says otherwise, programs are console applications that read input through `Scanner` or command-line arguments, validate everything they read, and print neatly formatted reports. You should test each program with normal, boundary, and invalid inputs. No problem requires file I/O, databases, or libraries outside the standard JDK.

How the Difficulty Progresses

The problems form one continuous sequence ordered by overall difficulty and integration, not by topic. Early problems are realistic applications built from fundamental constructs and a few well-designed classes. Later problems bring in more classes, richer object-oriented design, packages, stricter business rules, error handling, and shared-state correctness. The final problems are substantial desktop applications that integrate many parts of the syllabus at once. Because difficulty rises with scope and integration, you should attempt the problems in order, but you will meet familiar ideas again in new, harder combinations throughout.

Problem 1: Warehouse Carton Packing Planner

Scenario: A stationery distributor ships goods in a three-level packing hierarchy. Items are packed into boxes of 12, boxes into cartons of 12 (one gross of items), and cartons onto pallets of 10. A dispatch clerk needs a console tool that converts customer orders into packing instructions and order values.

Requirements:

- Read the number of order lines, then for each line read the SKU code, item name, units ordered, and unit price.
- For each line, compute the number of full pallets, cartons, boxes, and loose units using integer arithmetic.
- Apply a bulk discount to the line value: 8% if the line contains at least one full pallet, 4% if it contains at least one full carton but no full pallet, otherwise none.
- Print an aligned order table with the packing breakdown, discount, and line value, followed by the order total.
- Report the highest-value line and how many lines fell into each discount tier.
- After each order, ask whether another order should be processed, and repeat until the clerk declines.

Constraints:

- Between 1 and 20 lines per order; units between 1 and 1,000,000; unit price must be positive. Reject invalid values and re-prompt.
- Use arrays (not collections) to hold line data, and appropriate primitive types including `long` for quantity-based calculations.
- Use `for`, `while`, and `do-while` loops and a `switch` statement for the discount tier where each is natural.
- Split the program into well-named static methods; `main` should only coordinate.

Problem 2: Java Orientation Portal for a Training Institute

Scenario: A software training institute wants a console orientation tool that introduces new trainees to how Java evolved and why it was designed as it was.

Requirements:

- Store at least ten Java milestones (from the early Green/Oak project to recent long-term-support releases) as parallel arrays holding version, year, and a headline feature.
- Provide a menu to list the full timeline, show milestones within a year range supplied by the user, and look up a milestone by version.
- Describe each Java buzzword (simple, object-oriented, distributed, robust, secure, architecture-neutral, portable, interpreted, high performance, multithreaded, dynamic) and, for at least five of them, have the program perform a small live demonstration of the property (for example, reporting runtime platform properties, or the fixed sizes of primitive types).
- Include a ten-question quiz on the content that tracks the score and gives feedback on each answer.

- Include a short written note, supplied as a comment block or text printed by the program, distinguishing the JDK, JRE, and JVM and explaining bytecode.

Constraints:

- All historical facts must be verified against a reliable source; do not guess release years.
- Invalid menu choices and non-numeric input must be handled without the program terminating.
- Use a `switch`-driven menu, arrays, and at least five static methods.

Problem 3: Electricity Billing for a Distribution Company

Scenario: A power distribution company bills domestic, commercial, and agricultural consumers using category-specific slab tariffs. Billing clerks enter meter readings at the end of each cycle.

Requirements:

- For each consumer, read ID, name, category code (`D`, `C`, or `A`), previous reading, and current reading.
- Compute units consumed and the energy charge using cumulative slab rates specific to each category (define at least three slabs per category).
- Add a fixed charge per category, a 5% electricity duty on the energy charge, and a surcharge of 2% of the bill for each full month the previous bill is overdue (input by the clerk).
- Apply a minimum bill per category.
- Provide three overloaded `computeBill` methods that progressively accept units only, units plus category, and units plus category plus an overdue-month count.
- After processing all consumers, print a bill register with total revenue by category and the consumer with the highest bill.

Constraints:

- A current reading lower than the previous reading is a meter fault; flag the record and exclude it from revenue totals.
- Slab computation must be cumulative, not flat.
- Store records in arrays; the number of consumers is entered at run time (maximum 50).

Problem 4: Semester Result Statement Generator

Scenario: An examination cell produces result statements from marks passed on the command line, so the tool can be called from scripts.

Requirements:

- The first argument is the student name, the second is the roll number, and each remaining argument has the form `CODE:marks:credits`.
- Parse each subject token using `String` methods, then compute total marks, percentage, a credit-weighted grade point average using a grade-point table, a letter grade per subject, and an overall pass or fail remark.
- Implement an overloaded set of `calculate` methods, including a variable-argument version that accepts a name and any number of marks.
- Print a formatted statement and list any malformed tokens separately rather than silently ignoring them.
- If no subject tokens are supplied, print the message "No subjects entered".

Constraints:

- Marks must be 0 to 100 and credits 1 to 4. A subject with marks below 35 is a fail regardless of overall percentage.
- Duplicate subject codes in the arguments must be detected and reported.
- The program must handle any malformed argument without crashing.

Problem 5: Pharmacy Stock Item Management

Scenario: A pharmacy chain tracks medicine batches so it can sell stock safely, avoid expired medicines, and reorder in time.

Requirements:

- Design a `Medicine` class with batch number, name, manufacturer, quantity on hand, unit price, and expiry month and year.
- Provide a default constructor, a constructor taking name and price, and a full constructor. Reuse initialization logic through constructor chaining, and use `this` wherever parameters shadow fields.
- Provide validated setters that support method call chaining.
- Implement `sell(int qty)`, `restock(int qty)`, `isExpired(...)`, `valueOnHand()`, and two overloaded `applyDiscount` methods (percentage only, and percentage with a minimum quantity).
- A driver creates at least eight medicines and produces an expired-stock report, a low-stock report with a configurable threshold, and a total inventory valuation.

Constraints:

- All fields private; quantity can never become negative; price must be positive; expiry month 1 to 12.

- `sell` must refuse expired medicines and requests exceeding stock, and report the reason.
- Document the default values chosen by the default constructor.

Problem 6: Prepaid Metro Card System

Scenario: A metro rail operator issues prepaid cards that deduct zone-based fares at entry and exit gates.

Requirements:

- Design a `MetroCard` class with a unique card ID auto-assigned from a static counter (starting at 50001), balance, and the last ten trips.
- Hold a zone-to-zone fare table as a static two-dimensional array initialized in a static initialization block, which also prints a one-time "fare table loaded" message when the class loads.
- Provide overloaded `recharge` methods: one for an amount and one for an amount plus a promotional code that adds a bonus.
- Provide `swipeIn` and `swipeOut` methods that compute the fare from the entry and exit zones and deduct it.
- Provide static methods to report the total cards issued and to revise the fare between a pair of zones.
- A driver issues several cards, simulates journeys and recharges, and prints each card's statement.

Constraints:

- Card ID and issue year are `final`. Balance must not exceed 10,000 and must be at least the maximum fare to permit entry.
- A swipe-out without a matching swipe-in must be rejected. Only the latest ten trips are retained.
- Static methods may not use instance state.

Problem 7: Hotel Reservation Desk

Scenario: A boutique hotel's front desk records reservations and later amends or duplicates them for group bookings.

Requirements:

- Design a `Reservation` class with a unique reservation number (static counter), guest name, room type, number of nights, nightly rate, and an array of extra services.

- Provide default, parameterized, and copy constructors; chain constructors with `this(...)`.
- Provide chainable setters.
- A `Hotel` class holds up to 25 reservations. A `Reservation` registers itself with the hotel through a method that passes the current object.
- Implement an overloaded `bill` method (room only, room plus extras, and room plus extras plus a loyalty discount percentage).
- A copy used for a group booking must not share its extras array with the original.
- The driver demonstrates that modifying a copy leaves the original unchanged.

Constraints:

- Nights between 1 and 30; room types limited to a defined set of rates; reject duplicate guest and room-type bookings for the same reservation batch.
- The hotel must refuse registration when full.
- Provide a report of all reservations and the total expected revenue.

Problem 8: Hospital Ward Vitals Monitoring Service

Scenario: A hospital ward system records patients' vital signs several times a day, raises alerts, and updates readings from different parts of the application. The software team must be certain which updates are actually visible to callers.

Requirements:

- Model a `VitalsReading` (heart rate, systolic and diastolic blood pressure, temperature) and a ward of five patients, each with several readings.
- Implement service methods that: attempt to normalize a primitive heart-rate value; correct a reading by modifying the passed object; try to replace a passed reading by assigning a new object to the parameter; apply a uniform adjustment across an array of readings; and try to swap two patients' latest readings.
- Run each method on real ward data and print before and after states.
- Generate an alert report for readings outside clinical thresholds and a daily summary per patient.
- Include a short written findings section, printed by the program or supplied as comments, explaining what each experiment shows about Java's parameter-passing mechanism.
- Include an immutable reading type for archived data.

Constraints:

- Archived readings use `final` fields and cannot be modified after construction; use `final` parameters in at least two methods.
- Thresholds should be named constants.
- Every observation in the written findings must correspond to behavior your program actually exhibits.

Problem 9: Restaurant Order Billing

Scenario: A restaurant billing system groups ordered dishes into orders, applies taxes and service charges, and offers time-limited pricing.

Requirements:

- Design an `Order` outer class with table number, server name, and an order-wide tax rate.
- Define an inner (non-static) `OrderLine` class that represents a dish, quantity, and unit price and uses the enclosing order's tax rate when computing its tax.
- Define a static nested `ServiceChargePolicy` class holding charge rules by party size.
- Inside the billing method, define a local class that applies happy-hour pricing to eligible lines.
- Provide methods to add and remove lines, print an itemized bill, and compute the grand total.
- A driver creates several orders at different tables and prints all bills plus a day summary.

Constraints:

- Quantity at least 1; a dish may appear only once per order, so repeated additions increase the quantity.
- Inner class instances must be created through an enclosing order object.
- The driver must demonstrate, through output, how the inner class reads outer state and how the static nested class is created without an enclosing order.

Problem 10: Self-Service Kiosk Session Manager

Scenario: A railway information kiosk creates a session object for each passenger. Each session reserves a sizeable memory buffer, and operations staff want to observe how session objects are reclaimed.

Requirements:

- Design a `KioskSession` class with a unique session ID, start time, and a large internal buffer.
- Track created, explicitly closed, and finalized sessions using static counters.
- Provide an explicit `close` method for orderly resource release, and also override `finalize()` to log reclamation, as the syllabus requires.
- Write a load generator that creates thousands of sessions, drops references in different ways (nulling a reference, reassigning it, and letting a variable go out of scope), requests garbage collection, and reports memory use and counters at each stage.
- Compare a run in which sessions are explicitly closed with a run in which they are left to the collector.
- Include a note describing why `finalize()` is deprecated and why explicit release is the recommended design.

Constraints:

- The program must never depend on finalization having run; report finalized counts as observations only.
- Counter updates must remain consistent when `close` and `finalize` are both invoked on the same object.
- Memory figures must be obtained from the runtime and printed in a readable unit.

Problem 11: Multiplex Seat Map and Box Office

Scenario: A multiplex has three screens, each with a different seating layout, and a box-office clerk sells and cancels seats.

Requirements:

- Represent each screen's layout with a two-dimensional array in which front rows are shorter than back rows (jagged arrays), with labels such as A1 or C12.
- Provide a menu to display the seat map for a screen, book specific seats, book a number of adjacent seats in a chosen row, cancel seats, and view revenue.
- Use overloaded booking methods for specific seats versus a seat count.
- Price seats by row category (for example, premium, standard, economy), and compute the total for a booking.
- Produce an occupancy report per screen, with percentage occupied by category.

Constraints:

- Booking must be all-or-nothing: if any requested seat is unavailable or invalid, none is booked.

- Reject out-of-range rows and seats, and cancellations of seats that are not booked.
- Seat state must be stored in the arrays, not in collections.

Problem 12: Home Loan Planning Assistant

Scenario: A bank wants a console planner that helps customers understand monthly instalments, total interest, and the effect of part-payments before they apply for a home loan.

Requirements:

- Read principal, annual interest rate, and tenure in years.
- Compute the monthly instalment, total interest, and total payable.
- Print a year-wise summary showing principal repaid, interest paid, and closing balance.
- Support a part-prepayment scenario that shows the reduction in tenure or in instalment, as the customer chooses.
- Compare three lenders with different rates and processing fees and identify the cheapest overall.
- Provide overloaded `emi` methods (basic, with processing fee, and with processing fee and insurance).

Constraints:

- Principal at least 100,000; rate between 1% and 20%; tenure between 1 and 30 years.
- Use consistent rounding to the nearest paisa and show that the schedule reconciles to zero within one rupee.
- Use compound assignment, the conditional operator, explicit casting, and `Math` methods where they fit naturally.

Problem 13: Student Hostel Allotment Office

Scenario: A college hostel office allots rooms to students according to block, gender, and year, and handles transfers and vacating.

Requirements:

- Design `Student`, `Room`, and `HostelBlock` classes. A room has a fixed capacity and holds its current occupants; a block holds its rooms and is restricted to one gender.
- Provide operations to allot a student, vacate a student, transfer a student between rooms or blocks, and list occupants by room, block, and year.
- Keep a static count of total students currently accommodated and a static source of unique student allotment numbers.

- Provide a report of vacancy by block and the list of students still awaiting accommodation.

Constraints:

- Fields are private with controlled access; accessors must not expose internal arrays that would allow outside code to bypass the rules.
- Room capacity is `final`. Use `this` where appropriate.
- A student can occupy only one room at a time; gender and capacity rules must hold after every operation, including transfers.

Problem 14: Retail Chain Sales Dashboard

Scenario: A retail chain's head office reviews monthly sales across its branches against targets.

Requirements:

- Design a `Branch` class (name, city, annual target) and a `SalesLedger` holding monthly sales for each branch in a two-dimensional array.
- Read or generate twelve months of data for at least five branches.
- Compute branch totals, quarterly totals, best and worst month per branch, month-on-month growth percentages, and chain-wide totals.
- Flag branches below 90% of the target at the end of each quarter.
- Provide overloaded `average` methods (for a branch, for a branch and quarter, and for the chain).
- Include a static utility class containing formatting and percentage helpers.
- Print the results as clean tables.

Constraints:

- Sales figures must be non-negative; missing months are reported, not silently treated as zero.
- No collections; use arrays throughout.
- Utility methods must be static and independent of object state.

Problem 15: Hospital Staff Directory and Pay Processing

Scenario: A hospital maintains one directory for doctors, nurses, and administrative staff, each of whom is paid by different rules, and its payroll office must process them uniformly.

Requirements:

- Design a base `HospitalStaff` class with employee ID, name, department, and joining year, and subclasses `Doctor`, `Nurse`, and `AdminStaff`.
- A doctor's pay includes a base salary plus a per-consultation fee; a nurse's pay includes a base salary plus a night-shift allowance per shift; administrative staff receive a base salary plus a fixed allowance.
- Each subclass overrides the pay calculation and the display method, reusing the parent's behavior through `super`, and invokes the parent constructor with `super(...)`.
- Hold all staff in a single array of the base type and process it polymorphically to print pay slips.
- Produce a report of total pay per staff type and a count of staff per department.

Constraints:

- Parent fields are `protected` or private with accessors; justify your choice in a short comment.
- Using `instanceof` is permitted only for the per-type reporting, not for calculating pay.
- IDs are unique and assigned automatically.
- Reject negative consultations or shifts.

Problem 16: Campus Payroll Processor

Scenario: A university pays permanent faculty, contract lecturers, visiting experts, and interns using different pay rules but one common statutory deduction process.

Requirements:

- Create an abstract `CampusEmployee` class with an abstract method for gross pay and a concrete method for statutory deductions that subclasses cannot override.
- Implement `PermanentFaculty` (basic plus allowances), `ContractLecturer` (hourly rate times hours with a monthly cap), `VisitingExpert` (per-lecture fee), and `Intern` (fixed stipend, no deductions).
- Support overtime or extra-lecture payments where the category allows it.
- Process a mixed collection through the abstract type and print pay slips and a department-wise payroll summary.
- Report the highest-paid employee in each category.

Constraints:

- The abstract class must not be instantiable; subclass constructors must call the parent constructor.
- The deduction method must be `final`.

- Monetary amounts must not become negative; reject invalid hours and lecture counts.
- Do not copy rules between subclasses; shared behavior belongs in the parent.

Problem 17: Insurance Premium Engine

Scenario: An insurer sells life, health, and vehicle policies whose premiums depend on customer profile and claims history.

Requirements:

- Create an abstract `Policy` class with policy number, holder name, age, and sum insured, and an abstract premium calculation.
- Implement `LifePolicy`, `HealthPolicy` (with a family-size and pre-existing-condition loading), and `VehiclePolicy` (with vehicle age and a no-claim bonus).
- Each subclass computes its premium by first reusing the parent's base-premium method through `super`, then applying its own adjustments.
- Add a shared tax computation that no subclass may change.
- Support renewal, which recomputes the premium with updated claim history.
- Print a portfolio report of premium by policy type and the policies due for renewal.

Constraints:

- Age and sum insured must be within product limits that you define and document.
- Use dynamic method dispatch for portfolio processing; no type-checking code in the report.
- A policy's number is assigned once and never changes.

Problem 18: Online Payment Gateway

Scenario: An e-commerce company accepts payments through UPI, cards, net banking, and wallets, each with different limits, fees, and refund rules.

Requirements:

- Define a `PaymentMethod` interface with constants for the maximum transaction limit and the retry limit, plus methods to authorize and to compute the processing fee.
- Define a separate `Refundable` interface, and a `Verifiable` interface that extends it for methods needing identity verification.
- Implement `UpiPayment`, `CardPayment`, `NetBankingPayment`, and `WalletPayment`, where only suitable ones implement the additional interfaces.

- A `CheckoutService` accepts payments through the interface type and processes a batch of orders, printing a transaction report, total fees by method, and refunds issued.
- Include a short written comparison, as comments or printed text, of why these are interfaces rather than a base class and how this differs from an abstract class.

Constraints:

- Interface constants must be used in at least two implementing classes.
- A refund is allowed only for methods that implement `Refundable` and never exceeds the amount paid.
- Transactions above the interface limit are declined with a reason.

Problem 19: Logistics Fleet Maintenance and Tracking

Scenario: A delivery company manages trucks, vans, and bikes and needs to schedule servicing, track location, and estimate running costs.

Requirements:

- Define separate interfaces for tracking, servicing, and fuel-cost estimation, and an extended `FleetAsset` interface that combines them.
- Create an abstract base class implementing the common parts of `FleetAsset`, and concrete `Truck`, `Van`, and `DeliveryBike` classes that add type-specific behavior (load capacity, service interval, fuel efficiency).
- Record odometer readings and compute the next service due for each vehicle.
- A fleet manager processes the entire fleet through interface references to produce a service-due list, a fuel-cost estimate for a given route, and a utilization report.

Constraints:

- Each interface must be small and focused. At least one concrete class must implement more than one interface directly.
- Odometer readings can never decrease.
- Interface constants define service thresholds; document why constants belong in the interface.

Problem 20: Equipment Rental Registry

Scenario: A tools and equipment rental shop must recognize when two records refer to the same item, produce readable logs, and copy item records safely.

Requirements:

- Design `RentalItem` and subclasses `PowerTool`, `Scaffold`, and `Generator`, each with an asset tag, description, daily rate, and condition.
- Override `toString`, `equals`, and `hashCode` so that identity is determined by asset tag and type, and demonstrate the difference between `==` and `equals`.
- Implement a safe copying mechanism using the `Object` class's cloning support, ensuring that mutable fields are not shared.
- Keep a mixed inventory in an `Object` array. Process it using casts guarded by type checks, and use `getClass()` to print type information.
- Produce a registry report that detects duplicate entries using your `equals` method.

Constraints:

- `equals` and `hashCode` must be consistent.
- `toString` must be informative and never return null.
- Cloned objects must be independent of the original for any array or mutable field.
- Document which `Object` methods you overrode and why.

Problem 21: Online Examination Role and Permission Control

Scenario: An examination portal has students, invigilators, and administrators, and a security team insists that some behavior must never be modified by later developers.

Requirements:

- Design a user hierarchy (`PortalUser`, `Student`, `Invigilator`, `ExamAdmin`) with permission checks and an action log.
- Use a `final` class for an immutable access token and a `final` method for the authentication step in the base class.
- Use `private`, package-level, `protected`, and `public` members deliberately, and show in a short report which subclasses may legitimately see which members.
- Dynamic dispatch must route each role's action request (start exam, flag candidate, publish results) to the correct permission rule.
- Provide a security audit listing every denied action with the user and reason.

Constraints:

- Document in comments what the compiler would reject if someone tried to extend the final class or override the final method.
- Permission rules belong to the roles, not to a central `if` chain.
- Tokens expire after a configurable count of actions.

Problem 22: Public Library System in Multiple Packages

Scenario: A city library is building a system by separate teams, so its code must be split into packages with clear visibility boundaries.

Requirements:

- Create a package hierarchy for the domain model (books, members, loans), the services (issue, return, fine calculation), the utilities (validation and formatting), and the console application.
- Implement issue, return, renewal, and overdue fine processing across the packages.
- Demonstrate access protection across packages: use public, protected, and package-private members deliberately, and include a subclass in another package that uses a protected member.
- Use `import` statements appropriately.
- Provide build and run instructions that compile into a separate output directory, and run the program by setting the classpath, both through `CLASSPATH` and through the command-line option.

Constraints:

- The model package must not depend on the application package.
- Package-private classes must not be usable from outside their package.
- The instructions must work from a clean directory, and the source tree layout must match the package names.

Problem 23: Utility Company Mail-Merge Engine

Scenario: A utility company sends personalized notices for payment reminders, disconnection warnings, and tariff changes, using one template per notice.

Requirements:

- Accept templates with placeholders for customer name, account number, due amount, and due date.
- Build each personalized letter using `StringBuffer`, replacing placeholders in place.
- Use `String` methods for trimming input, case conversion, substring extraction, searching, and formatting.
- Validate email addresses and phone numbers using string operations, and report invalid records separately.
- Generate masked account numbers, show only the last four digits, and produce a short subject line truncated to a maximum length with a trailing marker.
- Print each letter and a batch summary of letters generated and records rejected.

Constraints:

- At least ten customer records, including malformed ones.
- An unknown placeholder in a template must be detected and reported, not left in the output.
- Do not use regular expressions; the objective is string manipulation with the prescribed classes.

Problem 24: Restaurant Invoice and Receipt Formatter

Scenario: A point-of-sale vendor prints fixed-width receipts for thermal printers and needs to know when building text through concatenation becomes a performance problem.

Requirements:

- Build a 40-column receipt (header, itemized lines with aligned columns, totals, and footer message) using `StringBuffer` and its `append`, `insert`, `delete`, `replace`, and `reverse` operations.
- Support centering and right-aligning text, wrapping long item names across lines, and inserting separator lines.
- Report buffer `length` and `capacity` at several stages, and show how pre-sizing the buffer changes capacity growth.
- Compare building receipts for a large number of orders using repeated `String` concatenation against `StringBuffer`, measuring elapsed time and presenting the results.
- Demonstrate string immutability, `==` versus `equals`, and `intern` using actual receipt content.

Constraints:

- Every line must be exactly 40 characters wide; overflowing text must be wrapped or truncated according to a documented rule.
- Timing comparisons must use enough repetitions to be meaningful, and your report must state its own limitations.
- Item names and quantities come from the user at run time.

Problem 25: Railway Fare Quote Command-Line Tool

Scenario: A railway enquiry system provides a command-line fare quoting tool used by agents' scripts.

Requirements:

- Accept arguments for source station code, destination station code, travel class, number of adult and child passengers, and optional concession codes.
- Parse and validate each argument using `String` methods and print a clear usage message when arguments are missing or malformed.
- Look up the distance between station pairs in an internal table, apply class multipliers, child discounts, concession percentages (senior, student, defense), a reservation fee, and a superfast surcharge where applicable.
- Print an itemized quote with per-passenger fare, taxes, and total.
- Reject quotes for unknown stations or identical source and destination.

Constraints:

- Argument order may vary for optional arguments; the tool must still interpret them correctly.
- Concessions cannot be combined unless your documented rules allow it.
- Case-insensitive input for codes; output in upper case.
- A maximum of six passengers per quote.

Problem 26: Airline Fare, Baggage, and Refund Engine

Scenario: An airline's booking software prices tickets in several fare classes, applies baggage rules, and calculates refunds when passengers cancel.

Requirements:

- Create an abstract `Ticket` with PNR, passenger name, flight number, base fare, and hours to departure, and subclasses `EconomyTicket`, `BusinessTicket`, and `CorporateTicket`.
- Define a `BaggagePolicy` interface (free allowance and excess-charge computation) and a `Refundable` interface with a refund calculation; implement them according to fare class, and make the cheapest economy fare non-refundable.
- Refund amounts depend on the time remaining before departure and the fare class.
- A booking desk handles a batch of passengers: it issues tickets, calculates baggage charges, and processes cancellations, using only abstract types and interfaces.
- Override `toString` and `equals` in `Ticket`, treating two tickets as equal when the PNR matches.

Constraints:

- Refund must never exceed the fare paid.
- A cancellation after departure is rejected.
- Baggage weights must be positive.

- No duplicate PNRs.

Problem 27: E-Commerce Cart and Promotion Engine

Scenario: An online store applies a changing mix of promotions to shopping carts, including one-off campaigns that marketing wants to add without changing existing classes.

Requirements:

- Create a `Cart` class containing an inner class `CartItem` (product, quantity, unit price, category) that accesses cart-level data such as the customer's membership tier.
- Define a `PromotionRule` interface and implement several rules (percentage off a category, buy-three-get-one, minimum-spend discount, membership discount).
- Define a static nested `ShippingCalculator` computing shipping from weight and destination zone.
- Support campaign-specific rules created as anonymous classes within the campaign setup code.
- Apply all eligible promotions, with a documented rule for preventing double discounting of an item, and print an itemized invoice with savings per rule.

Constraints:

- Adding a new promotion must not require changing `Cart`.
- Local variables used in anonymous classes must be used correctly under Java's capture rules.
- Discounts must not make any line negative, and total savings must be reported.

Problem 28: Retail Banking Account Management

Scenario: A retail bank offers savings, current, and fixed-deposit accounts, along with loans to account holders, and needs a coherent object model before introducing new products.

Requirements:

- Define interfaces `InterestBearing` (with a constant for the base rate and an interest method) and `Loanable` (with an EMI calculation and eligibility check), as well as an abstract `Account` class.
- Implement `SavingsAccount` (minimum balance, withdrawal limit), `CurrentAccount` (overdraft facility), and `FixedDeposit` (term, premature-closure penalty), with the appropriate interfaces.
- Maintain a `Customer` holding multiple accounts, and a transaction log per account.

- Support deposits, withdrawals, transfers, monthly interest posting, and statement generation.
- Override `toString` and `equals` where it adds value, and process accounts polymorphically.
- Operations return status codes or boolean results with reasons, rather than throwing exceptions in this version.

Constraints:

- Account numbers are unique and generated by the bank.
- Balances must respect each product's rules after every operation.
- A transfer must either fully succeed or leave both accounts unchanged.
- Interest rates read from interface constants may be overridden only through documented product rules.

Problem 29: ATM Cash Dispenser with Fault Handling

Scenario: An ATM controller must remain stable regardless of bad input, hardware limits, and cash-cassette problems.

Requirements:

- Simulate a cash dispenser holding notes of several denominations, with a customer menu for balance enquiry, withdrawal, and PIN change.
- Dispense using available denominations and report when exact change cannot be made.
- Handle at least these built-in exceptions in meaningful places: arithmetic errors, invalid number input, malformed numeric text, out-of-range array access, and a null reference from an uninitialized card record.
- Use multiple `catch` clauses (ordered correctly), a `finally` block that always records the session in an audit log, and `throw` and `throws` for propagating errors between layers.
- Lock the card after three incorrect PIN attempts.

Constraints:

- The program must not terminate on any user input; each failure produces a customer-friendly message and a technical audit entry.
- A failed withdrawal must not alter the balance or the cassette contents.
- Withdrawal amounts must be multiples of the smallest denomination and within the daily limit.

Problem 30: University Course Registration Rules Engine

Scenario: A university's registration system must enforce prerequisites, seat limits, credit limits, and clash rules, and give students precise reasons when registration fails.

Requirements:

- Model students, courses (with prerequisites, seats, credits, and timetable slots), and a registration service.
- Define a user-defined checked exception hierarchy with a common base (for example, for registration failures) and specific subclasses for each rule: missing prerequisite, no seats, credit limit exceeded, timetable clash, and student not eligible.
- Methods should declare and propagate the appropriate exceptions using `throws`, and the caller should handle them with specific `catch` clauses.
- Registration for a whole semester plan is processed course by course, enrolling the valid courses and reporting every rejected one with the exact reason.
- Produce a final registration summary and a list of waitlisted requests.

Constraints:

- Exceptions must carry useful data (student, course, and the rule violated), not just a message.
- A failed registration must leave the student's and the course's state unchanged.
- Do not use exceptions for ordinary control flow that a simple condition would express more clearly.

Problem 31: Prescription Validation Pipeline

Scenario: A hospital pharmacy validates each prescription through several layers (format check, patient lookup, drug-interaction check, stock reservation) and must always release reserved stock, even when a later layer fails.

Requirements:

- Implement the four validation layers as separate methods, with an outer coordinator that processes a batch of prescriptions.
- Use nested `try` blocks so that each layer handles the failures it can recover from and passes the rest outward.
- Wrap lower-level failures in higher-level user-defined exceptions that preserve the original cause, and rethrow where appropriate.
- Use `finally` to release stock reservations and write an audit line for every prescription, and show through the audit order that `finally` runs in the right sequence for normal, handled, and rethrown cases.

- Distinguish checked and unchecked failures in your design, and use at least one of each.
- Print a batch report with each prescription's outcome and, for failures, the full chain of causes.

Constraints:

- Catch clauses must be ordered so that no catch is unreachable.
- Stock must be reserved and released exactly once per prescription, whatever happens.
- A failure in one prescription must never prevent the rest of the batch from being processed.

Problem 32: Payroll Import Data Validator

Scenario: A payroll team imports employee records from text lines exported from another system. The data contains many kinds of defects, and the team wants a precise error classification rather than a generic failure message.

Requirements:

- Embed or accept at least twenty text records in the form `id,name,department,basic,joiningDate,managerId` with a variety of defects: missing fields, non-numeric amounts, negative salaries, empty names, impossible dates, a missing manager, and over-long text.
- Parse and validate each record, allowing built-in exceptions (such as number-format, array-index, string-index, null-pointer, arithmetic, class-cast, and illegal-argument exceptions) to occur where genuine, and handle them by class.
- For each failure, record the exception class, message, and the offending record.
- Produce a summary of counts by exception class and a list of clean records.
- Classify each exception that appeared as checked, unchecked, or an error, and explain the classification in the report.

Constraints:

- Do not suppress exceptions by checking everything in advance; some defects must be detected through the exception mechanism, and you should state which ones and why.
- Never catch a broader type than necessary.
- At least one custom exception must be used for a business rule.

Problem 33: Library Catalogue and Loan Desk

Scenario: A college library wants a loan desk application that keeps its catalogue, members, and active loans in memory and cleans up overdue loans each day.

Requirements:

- Model books, members, and loans, and keep each group in a suitable collection. Use `ArrayList` for the catalogue and `LinkedList` for the daily returns queue, and justify the choice in a comment.
- Support adding titles, searching by title or author fragment, issuing and returning books, renewing loans, and listing a member's loans.
- Implement an end-of-day overdue processing step that uses an `Iterator` to scan all loans, compute fines, and remove loans that were returned.
- Provide a report of loans per member and the books never borrowed.

Constraints:

- Removing elements during a traversal must be done safely through the iterator, never by modifying the collection directly.
- A member can hold at most five loans; an unavailable book cannot be issued.
- Declare variables using collection interface types where appropriate.

Problem 34: Student Enrollment Registry

Scenario: A department keeps track of which students are enrolled in which courses and wants quick answers about overlaps and gaps.

Requirements:

- Use a `Map` from course code to the set of enrolled students and a second map from student to the courses taken, and keep the two views consistent.
- Use `HashSet`, `LinkedHashSet`, and `TreeSet` where their ordering behavior matches a reporting need, and `HashMap` and `TreeMap` likewise.
- Support enrolling, dropping, listing courses alphabetically, finding students enrolled in all of a given set of courses, students enrolled in at least one, and students in exactly one.
- Make `Student` usable as a set element and map key by implementing `equals` and `hashCode` correctly.
- Iterate over maps and sets using `Iterator` for the cleanup of a course that is cancelled.

Constraints:

- Duplicate enrollment attempts must be detected using the collection behavior, not manual loops.
- Each course has a capacity; enrolling beyond it is refused.
- Prove with a short test scenario that equal students are treated as one entry and that two views stay in sync after drops.

Problem 35: Outpatient Token and Triage Manager

Scenario: A hospital's outpatient department issues tokens, serves routine patients in arrival order, but must serve emergency and senior patients earlier.

Requirements:

- Maintain waiting patients in standard queue-type collections: a plain `Queue` for routine tokens, a `PriorityQueue` ordered by a documented urgency rule for triaged patients, and a `Deque` for patients recalled from the consultation room.
- Issue tokens, call the next patient (applying your documented policy among the three lines), mark a patient as absent and re-queue once, and cancel tokens.
- Support a doctor-wise view of the lines and a live display of waiting counts and average wait.
- Use an `Iterator` to produce a snapshot report without disturbing the queues.

Constraints:

- The priority rule must be explicit and deterministic, including ties.
- A patient who is absent twice is removed and logged.
- You must not implement queue behavior yourself; rely on the standard collection classes.

Problem 36: Typed Hospital Record Repository

Scenario: A hospital information system stores patients, doctors, and appointments, and its developers want one reusable, type-safe repository instead of three separate copies of the same code.

Requirements:

- Define a `Record` interface with a unique key, and implement it for `Patient`, `Doctor`, and `Appointment`.
- Write a generic `Repository<T>` class, bounded to records, that supports add, find by key, update, delete, list all, and filtering using a condition supplied by the caller.
- Prevent duplicate keys and report missing records through user-defined exceptions.

- Use the repositories together: book an appointment only if both the patient and the doctor exist, and print a day schedule per doctor.
- Demonstrate type safety: show in comments what the compiler rejects when someone tries to store the wrong record type.

Constraints:

- Use generics throughout; no raw types and no casting in client code.
- Repository storage must be an appropriate collection, hidden behind its public methods.
- Returned collections must not allow callers to alter repository internals.

Problem 37: Inventory Analytics Utility Library

Scenario: A warehouse analytics team wants a small utility library usable with products, suppliers, and shipments without being rewritten for each one.

Requirements:

- Write generic methods that: identify the maximum and minimum by a natural order, sum or average a numeric attribute, group items by a key into a map, print any array or collection in a standard layout, and swap or reverse positions in an array.
- Apply them to products (by price and stock), suppliers (by rating), and shipments (by weight), using bounded type parameters where the operation needs comparison or numeric behavior.
- Add a generic pair class holding two related values (for example, a product and its stock level) and a generic method that builds pairs from two lists.
- Print an analytics report that exercises every method on at least two different types.

Constraints:

- No method may be duplicated for different element types.
- Methods must tolerate empty input without failing, with a documented result.
- Type bounds must be as strict as the operation needs and no stricter.

Problem 38: Warehouse Stock Ledger

Scenario: A distribution warehouse records every stock movement (receipt, issue, transfer, adjustment) and must prevent negative stock while giving auditors a complete history.

Requirements:

- Write a generic `Ledger<T>` class that stores movements for an item type `T`, maintains the running balance per item in a map, and keeps ordered movement history in a list.
- Define user-defined exceptions for insufficient stock, unknown item, and invalid movement.
- Support receiving stock, issuing stock to a department, transferring between two warehouses, and adjusting for damage or counting differences.
- Provide audit reports: movement history for an item, current balances, items below reorder level, and the largest movements.
- Use an `Iterator` to build reports and to purge entries older than a retention limit while keeping balances correct.

Constraints:

- A transfer must be atomic across two ledgers: both change or neither does.
- Balances must always reconcile with the sum of the movement history.
- Exceptions must identify the item, quantity requested, and quantity available.

Problem 39: Telecom Billing Across Packages

Scenario: A telecom operator's billing team splits its code into several packages and processes call detail records into monthly bills for prepaid, postpaid, and corporate customers.

Requirements:

- Organize the code in separate packages for the model (plans, subscribers, call records), billing logic, and the console application.
- Create an abstract plan type and concrete plans with different free allowances, per-minute and per-message rates, and roaming charges.
- Process call records supplied as text lines (caller, callee, start time, duration, type, and region), parsing them with `String` operations and building report text with `StringBuffer`.
- Compute each subscriber's bill, with itemized usage, taxes, and taxes on roaming, and print a billing run summary with revenue by plan type.
- Use collections to group records by subscriber.

Constraints:

- Package-private members must be used where they help, with a protected member used by a subclass in another package.
- Malformed records are reported with line numbers and skipped; they must not stop the run.

- Call durations must be positive and not exceed a daily maximum.

Problem 40: Application Log Analyzer

Scenario: An operations team receives batches of application log lines and wants a quick summary before opening the full logs.

Requirements:

- Accept log lines in the form `timestamp level module message`, supplied by the user or from an internal sample of at least thirty lines.
- Parse each line using `String` methods and report malformed lines separately.
- Use collections to count messages by level, find the most error-prone modules, list the distinct error messages with their occurrence counts, and show errors in a given time window.
- Mask sensitive values (such as card numbers and email addresses) in messages using `StringBuffer`, and print a sanitized report.
- Provide an interactive query menu so the analyst can filter by level, module, and keyword.

Constraints:

- Messages may contain separators; the parsing rule must handle them without losing text.
- Keyword search must be case-insensitive.
- Use `Map`, `List`, and `Set` where each is most appropriate, and traverse at least one using an `Iterator`.

Problem 41: Bank Statement Reconciliation Engine

Scenario: A company's finance team reconciles its internal ledger entries against a bank statement to find missing, duplicate, and mismatched transactions.

Requirements:

- Model ledger entries and statement lines, each with a reference, date, amount, and description, and define a common interface used by both.
- Match entries by reference and amount using collections, and classify results as matched, amount mismatch, missing in the bank, missing in the ledger, or duplicate.
- Define user-defined exceptions for malformed statement lines and unreconcilable differences, handled so that one bad record never stops the run.

- Produce a reconciliation report with totals for each class, the net unreconciled amount, and a list of items needing follow-up.
- Use `StringBuffer` to build the printable report.

Constraints:

- Equality of entries must be defined in terms of the matching rule, and implemented consistently.
- Amounts must be handled without floating-point rounding surprises; document your approach.
- The ledger and statement collections must not be modified by the reporting step.

Problem 42: Online Examination Engine

Scenario: A coaching institute runs timed online tests with a question bank, different question formats, and an automatic marking scheme.

Requirements:

- Define an abstract `Question` with subclasses for single-choice, multiple-choice, and numeric-answer questions. Each marks answers differently, with negative marking for wrong single-choice answers and partial credit for multiple-choice answers.
- Keep the bank in collections grouped by subject and difficulty, and build a randomized paper with a required mix.
- Simulate an examinee attempting the paper with answers typed at the console, with the ability to skip and revisit questions.
- Define exceptions for an invalid answer format, a duplicate submission, and a paper-assembly failure when the bank is too small.
- Produce a score card with section-wise marks, accuracy, and a list of incorrect answers with the correct ones.

Constraints:

- No question may appear twice in a paper.
- Marking rules must be polymorphic; the scoring code must not test question types.
- A paper must be reproducible from a seed value supplied by the user.

Problem 43: Railway Reservation with Waiting List

Scenario: A railway reservation service allocates berths on trains with several coach classes, maintains a waiting list, and promotes waiting passengers when someone cancels.

Requirements:

- Model trains, coaches, berths, passengers, and bookings. Each train runs on specific days and has different quotas.
- Support booking one or more passengers on a journey, confirming berths according to preferences where possible, and otherwise placing the booking on a waiting list.
- On cancellation, release the berths and promote the earliest eligible waiting bookings, following documented rules for partial confirmations.
- Use user-defined exceptions for invalid journeys, sold-out classes, and cancellations of non-existent bookings.
- Provide a chart-style report of berth occupancy by coach and a passenger manifest.

Constraints:

- The booking rules must be implemented with the standard collection classes.
- A passenger cannot hold two bookings for overlapping journeys on the same train and date.
- Waiting-list promotion order must be deterministic and documented.

Problem 44: Insurance Claims Workflow

Scenario: An insurer handles claims through registration, document verification, assessment, and approval, with approval authority depending on the claimed amount.

Requirements:

- Model claims with a status that moves through the workflow stages, and enforce that only legal transitions are allowed.
- Define a `ClaimHandler` interface and implement a chain of approvers (assistant, officer, manager, committee) with escalating amount limits.
- Keep active, approved, and rejected claims in suitable collections and move claims between them.
- Define user-defined exceptions for illegal transitions, missing documents, and claims over the policy limit.
- Place the model, workflow, and application code in separate packages.
- Produce a pipeline report showing counts and amounts at each stage, the longest-pending claims, and the approver workload.

Constraints:

- Rejected claims must carry a reason code that is mandatory at rejection time.
- A claim must not skip stages.

- Status values must be defined as named constants, with no unexplained literals.

Problem 45: Shared Print Spooler Simulation

Scenario: A department's shared printers accept jobs from many users at once. The IT team wants a simulation to understand thread behavior and shared-counter problems before building the real spooler.

Requirements:

- Model users submitting print jobs, each job with an owner, page count, and priority, and printers that process jobs.
- Implement job submitters by extending `Thread` and printers by implementing `Runnable`, and use both approaches deliberately.
- Maintain shared statistics (total pages printed and jobs completed per printer) updated by all printer threads.
- Observe and report each thread's life-cycle states (new, runnable, timed-waiting, blocked, terminated) at several points using sleep, join, and state queries, with a clear log.
- Run the statistics first without any protection and show that the totals can be wrong over many runs, then correct the design so that totals are always exact.
- Demonstrate priorities and report whether they changed the order of completion.

Constraints:

- The main thread must wait for all workers to finish before printing final statistics.
- The corrected version must give exact totals in every run, verified against the known total of submitted pages.
- Do not rely on sleep durations for correctness.

Problem 46: Concurrent Cinema Seat Booking

Scenario: A cinema's online portal and its box-office counters sell seats for the same show at the same time. Double-booking a seat would be a serious service failure.

Requirements:

- Model a show with a seat layout and booking requests for one or more seats, each from a separate customer thread.
- Launch a large number of customer threads from several channels (web, mobile, counter), each attempting bookings of random seat sets.
- Provide synchronized booking and cancellation operations that ensure a seat is never sold twice and that multi-seat bookings are all-or-nothing.

- After all threads finish, verify consistency: every seat has at most one owner, every successful booking appears exactly once, and totals match the revenue.
- Report successful and failed bookings per channel.

Constraints:

- Synchronization granularity must be justified: explain whether you lock the whole show or parts of it.
- A failed booking must leave no seats held.
- The consistency check must be performed by code, not by eye, and must actually be capable of detecting a double booking.

Problem 47: Online Fund Transfer Simulator

Scenario: A bank's payments system processes many simultaneous transfers between customer accounts, and the total money in the bank must never change.

Requirements:

- Model accounts with balances and a `Transfer` task executed by worker threads, each moving money from one account to another.
- Run thousands of random transfers across a small set of accounts using several threads.
- Provide deposit, withdrawal, and transfer operations that are thread-safe, including transfers where the same pair of accounts is used in opposite directions at the same time.
- Use both synchronized methods and synchronized blocks where each fits, and explain each choice.
- After the run, verify that the total balance is unchanged and no balance is negative, and report the number of rejected transfers for insufficient funds.

Constraints:

- The program must not deadlock, however the random pairs fall; state your strategy for choosing the order of acquiring the accounts.
- A transfer is atomic with respect to every other operation on the two accounts.
- A self-transfer must be rejected.

Problem 48: Order Dispatch Board with Producers and Consumers

Scenario: An e-commerce fulfillment center receives orders from several order-entry operators and dispatches them via a small number of packing stations. The dispatch board holds a limited number of pending orders.

Requirements:

- Implement an `OrderBoard` class with a fixed capacity, shared by producer threads (operators) and consumer threads (packers).
- Producers must wait when the board is full; consumers must wait when it is empty. Use inter-thread communication with `wait`, `notify`, or `notifyAll`, not polling.
- Orders have an ID, priority, and item count. Packers take the next order according to a documented rule, and packing time depends on the item count.
- Implement an orderly shutdown: operators finish, then packers drain the board and stop.
- Report orders produced, packed, and the maximum board occupancy, and show that none were lost or processed twice.

Constraints:

- Use a standard collection inside the board, with the board providing the coordination.
- No busy waiting.
- The program must terminate cleanly and print final statistics after every order has been packed.

Problem 49: Inter-Warehouse Transfer Deadlock Study

Scenario: A logistics company noticed that its nightly inventory transfer job occasionally freezes. The engineering team wants a reproducible demonstration and a verified fix.

Requirements:

- Model warehouses holding stock and transfer tasks that lock a source and a destination warehouse before moving stock.
- Build a scenario in which two or more transfer threads can reliably deadlock, and demonstrate it with a watchdog thread that detects the frozen state by examining thread states and reports which threads are stuck and which resources they hold or await.
- Provide a corrected implementation in which the same workload always completes, and compare the two runs.
- Write a short analysis, as printed text or comments, naming the four conditions for deadlock and identifying which one your fix removes.
- Extend the scenario to three warehouses in a cycle to show that the fix generalizes.

Constraints:

- The deadlock demonstration must terminate (for example, by the watchdog ending the run) rather than hang the program.

- The corrected version must handle both directions between the same pair.
- Stock totals must reconcile after the corrected run.

Problem 50: Railway Ticket Counter Simulation

Scenario: A railway station has several ticket counters and a shared pool of arriving passengers. Station management wants to know how many counters are needed for acceptable waiting times.

Requirements:

- Generate passengers with random arrival times and service needs, and place them in a shared waiting area served by counter threads.
- Run the simulation with a configurable number of counters, using threads that take the next waiting passenger, serve them for a time based on the request, and update shared statistics.
- Track average and maximum waiting time, passengers served per counter, and counter idle time.
- Support closing a counter during the run: the counter finishes the current passenger and stops, and another counter continues the work.
- Run the same workload with different counter counts and print a comparison table.

Constraints:

- Statistics must be exact despite concurrent updates; verify that the passengers served equal the passengers generated.
- Use thread interruption or an equivalent coordinated approach, not forced termination, to close a counter.
- Use `join` so the final report is only printed after every counter has stopped.

Problem 51: Parallel Customer Statement Generator

Scenario: A bank generates monthly statements for hundreds of thousands of customers. The batch must be divided among several workers, some customers will have bad data, and a failure must never lose the rest of the batch.

Requirements:

- Create a fixed number of worker threads, each implementing `Runnable`, and divide the customer list among them in a documented way.
- Each worker generates a statement for every assigned customer, building text with `StringBuffer`, and records the outcome.

- Use user-defined exceptions for data errors (missing address, negative closing balance with no overdraft, inconsistent transactions). Workers must catch and record them, and continue.
- Collect results into thread-safe shared collections and produce a final report listing statements generated, failures with reasons, and elapsed time per worker.
- Include a thread-safe global counter of processed customers that other threads can read as a progress indicator.

Constraints:

- Results from all workers must appear exactly once in the final report.
- An uncaught exception in a worker must be reported, not silently lost.
- The main thread must not read shared results before all workers are finished.

Problem 52: Generic Message Mailbox for a Notification Service

Scenario: A notification platform sends e-mails, SMS messages, and app alerts, and wants one reusable thread-safe mailbox that can carry any one message type safely between producers and delivery workers.

Requirements:

- Write a generic `Mailbox<T>` class with a bounded capacity, blocking `post` and `take` operations using inter-thread communication, and a method to close it.
- Define message types (`EmailMessage` , `SmsMessage` , `PushAlert`) and create a separate typed mailbox for each, with several producer threads (for example, billing, marketing, and security systems) and delivery worker threads for each type.
- Messages carry a priority, and urgent security alerts must be delivered before routine ones.
- Add a generic method that drains a mailbox into a list for end-of-day reporting.
- Closing a mailbox must release all waiting threads cleanly and prevent further posting.

Constraints:

- Type safety must be enforced by the compiler; no casts in client code.
- No message may be lost, delivered twice, or delivered after the mailbox has been closed and drained.
- Threads that are waiting when the mailbox closes must terminate rather than hang.

Problem 53: Operating Theatre Resource Coordination

Scenario: A hospital's operating theatres share a limited set of equipment kits and a team of anesthetists, and each surgery requires several resources at once.

Requirements:

- Model surgeries (each needing one theatre, one surgeon, one anesthetist, and one or more equipment kits), and a resource manager that grants or withholds resources.
- Run each surgery as its own thread that requests its full resource set, waits until it can be granted, performs the surgery for a duration, then releases everything.
- Use inter-thread communication so that waiting surgeries are woken when resources are released.
- Prioritize emergency surgeries over elective ones without starving the elective ones indefinitely; document your policy.
- Log each surgery's request, grant, start, and finish, and print a utilization summary for every resource type.

Constraints:

- A surgery must receive all its resources together or none (to avoid partial holding).
- The log must make it possible to verify that no resource was ever used by two surgeries at once; include a verification step in the program.
- The program must finish and not deadlock under heavy contention.

Problem 54: Live Auction Bidding Engine

Scenario: An online auction house runs time-limited auctions in which many bidders place bids simultaneously and the highest bid at closing time wins.

Requirements:

- Model auction items with reserve prices and minimum bid increments, bidders, and bids.
- Run each bidder as a thread that places bids at random intervals, sometimes responding to being outbid.
- Run a timer thread that closes each auction at its end time, and a bid-processing component that accepts only valid bids and guarantees that the recorded winner is the highest valid bid.
- Include a closing-minute rule that extends the deadline when a bid arrives near the end, up to a documented maximum number of extensions.

- Define user-defined exceptions for rejected bids (too low, auction closed, bidder blocked), and produce an auction report with the bid history, winner, and rejected-bid counts per reason.

Constraints:

- A bid arriving after closing must be rejected even if it was started before closing.
- The final winning bid must equal the highest accepted bid in the history; verify it in code.
- Bidders cannot bid on items they list.

Problem 55: Smart Parking Lot Gate Controller

Scenario: A shopping mall's multi-level parking lot has several entry and exit gates operated simultaneously, and drivers waiting at a full lot should be admitted as soon as a space is freed.

Requirements:

- Model parking levels with capacities for cars, bikes, and reserved bays, vehicles, and gate controllers.
- Run each vehicle as a thread that arrives, requests a space of the suitable type, parks for some time, and then leaves; run several gates concurrently.
- If no suitable space is free, the vehicle waits (up to a configurable limit) and is admitted when one becomes available, otherwise leaving unserved.
- Compute the parking fee on exit with a time-based tariff and collect revenue safely.
- Print a live occupancy log and a final report with vehicles served, vehicles turned away, revenue, and peak occupancy per level.

Constraints:

- The occupancy must never exceed capacity or drop below zero, verified by the program.
- Waiting threads must be notified when space is freed; no polling.
- A vehicle's departure must not be possible before its arrival is recorded.

Problem 56: Ride-Hailing Dispatch Center

Scenario: A ride-hailing company matches rider requests with nearby available drivers in several city zones, with requests expiring if unmatched for too long.

Requirements:

- Model zones, riders, drivers, and trips. Each zone holds waiting requests and available drivers in collections.
- Run rider-request generators and driver threads concurrently. Drivers become available, are matched to a waiting request in their own zone (or a neighboring zone after a documented delay), drive for a simulated duration, and become available again.
- Requests expire after a maximum waiting time and are reported as unserved.
- Compute fares with a surge multiplier based on the ratio of waiting requests to available drivers in a zone.
- Print a summary of trips completed, requests expired, driver earnings, and average wait per zone.

Constraints:

- A request must be matched to at most one driver and a driver to at most one request at a time.
- Use timed waiting rather than polling.
- Generic classes or methods must be used for at least one reusable component (for example, the per-zone pool).

Problem 57: Multi-Branch Bank End-of-Day Processing

Scenario: A bank runs an end-of-day batch at each of its branches. The batch posts interest, applies charges, settles inter-branch transfers, and then produces branch statements and a consolidated head-office report.

Requirements:

- Organize the code into packages for the account model (an abstract account with savings, current, and loan subclasses and shared interfaces), the batch processing logic, and the application.
- Run each branch's batch in its own thread. Inter-branch transfers settle through shared settlement accounts accessed by multiple branch threads.
- The head-office reporting thread must wait until every branch has finished its posting phase (use coordination, not a fixed delay) before consolidating.
- Use user-defined exceptions for account-level failures; a failure on one account must not stop its branch or other branches.
- Use collections and at least one generic class (for example, a typed result collector).
- Print per-branch summaries and a consolidated report showing total deposits, interest posted, charges levied, and failures.

Constraints:

- The settlement accounts must remain consistent regardless of thread interleaving.
- The consolidated totals must equal the sum of branch totals, verified by code.
- The head-office thread must not start consolidation early and must not wait forever if a branch thread fails unexpectedly.

Problem 58: Library Reservation Hold and Expiry Service

Scenario: A university library lets members reserve a title that is currently on loan. When a copy is returned it is held for the first member in the reservation list for 48 simulated hours, after which the hold expires and the next member is offered it.

Requirements:

- Model titles, copies, members, loans, and reservations. Returns, pick-ups, and new reservations arrive from several concurrent member and staff threads.
- When a copy is returned, place it on hold for the next reserved member and notify them.
- Run a background expiry thread (with a clock scaled so that 48 hours pass in a few seconds) that releases expired holds and offers the copy to the next member, or returns it to the shelf if nobody is waiting.
- Support members cancelling reservations and picking up held copies at any moment, including right at expiry.
- Define user-defined exceptions for rule violations (hold expired, not first in queue, limit exceeded).
- Print an event log and a final consistency report.

Constraints:

- A copy must never be both held and lent, or held for two members.
- A pick-up racing with expiry must have exactly one winner, with the loser receiving a clear message.
- The expiry thread must stop cleanly at the end of the simulation.

Problem 59: Employee Onboarding Desktop Form

Scenario: A company's HR department needs a desktop form through which staff register new hires and review the day's registrations.

Requirements:

- Build the form in a `JFrame` with labels (including a company logo shown as an icon), text fields for name, e-mail, phone, and date of joining, a department and designation entry, and buttons for Register, Clear, and Remove Selected.

- Validate every field when Register is pressed, show clear error messages, and place the focus on the first invalid field.
- On success, create an `Employee` object, assign an employee code automatically, and append a formatted entry to a scrollable text area that shows the day's registrations.
- Handle events through the delegation event model, using at least one named listener class, one inner class, and one anonymous class, and describe in comments which event source, event object, and listener are involved in each case.
- Display a running count of registrations in a status label.

Constraints:

- Validation and employee creation belong in non-GUI classes; the GUI classes only call them.
- Duplicate e-mail addresses must be refused.
- Invalid input must never throw an unhandled exception into the event thread.

Problem 60: Hotel Booking Quote Calculator

Scenario: A hotel's reservation desk wants a desktop tool that shows a live price as the agent changes options during a phone call.

Requirements:

- Provide a combo box for the city or branch, radio buttons for room type (standard, deluxe, suite), check boxes for add-ons (breakfast, airport pickup, spa pass, late checkout), and text fields for the guest name and number of nights.
- Update a bill summary panel immediately whenever any selection changes, showing room charges, add-ons, tax, and the total.
- Apply seasonal pricing and a long-stay discount based on documented rules.
- Use both item events and action events, and a separate `BillCalculator` class that contains all pricing logic.
- A Confirm button validates the form, and adds the booking to a list of confirmed bookings displayed in the window.

Constraints:

- Radio buttons must be mutually exclusive through proper grouping.
- The pricing class must not depend on Swing classes.
- Nights must be from 1 to 30; invalid values are flagged next to the field without a disruptive dialog on every keystroke.

Problem 61: Student Marks Register

Scenario: A department's examination staff wants to enter and review marks for each course in a spreadsheet-like desktop window.

Requirements:

- Display students and subject marks in a `JTable` inside a scroll pane, supported by your own table model backed by a collection of student records.
- Computed columns (total, percentage, grade, and pass or fail) must update immediately after any mark is edited.
- Support adding and deleting students, sorting the view by a chosen column in either direction, and a summary panel with class average, highest scorer, and pass percentage.
- Reject invalid marks when the cell edit is committed and revert to the previous value with an explanation.
- Provide an export action that places a formatted text report in a text area built using `StringBuffer`.

Constraints:

- The table model must be the single source of truth; the summary panel updates through the model's change notifications.
- Roll numbers are unique and not editable once created.
- Marks must be between 0 and the subject maximum.

Problem 62: Interior Floor-Plan Sketcher

Scenario: An interior design studio wants a lightweight tool to sketch room layouts with furniture symbols and see the areas immediately.

Requirements:

- Create a custom drawing component by extending `JComponent`, which paints a grid, walls, doors, and a palette of furniture rectangles and circles.
- Support mouse press, drag, and release to draw walls and place furniture, mouse movement to display the current coordinates, and keyboard shortcuts to switch tools, delete a selected item, and nudge it with the arrow keys.
- Use a mouse listener or motion listener with adapter classes where only some methods are needed, and explain in comments the event classes involved.
- Maintain a model of the drawn items in a collection, so that the component can repaint from the model whenever needed.

- Show the area of the room and the percentage covered by furniture in a status bar, and provide a Clear option with confirmation.

Constraints:

- Drawing logic and the model must be separate classes.
- Items must not be placed outside the room boundary or overlap existing furniture; refuse with a visible message.
- The component must repaint correctly after window resizing.

Problem 63: Hospital Front-Desk Workstation

Scenario: A hospital's front-desk staff manage patient registration, appointment booking, and billing from one workstation and switch between tasks continually during a shift.

Requirements:

- Use a tabbed pane with at least three tabs: patient registration (forms with text fields, radio buttons, and a combo box), appointment booking (doctor selection, date and slot choices, and a table of the day's appointments), and billing (itemized charges, payment mode, and a printable summary).
- Build on a generic `Repository<T>` and domain classes (patients, doctors, appointments, and bills), with a custom exception hierarchy for business-rule failures, which the GUI must present as understandable messages.
- Selecting a patient in one tab should make that patient the current context in the other tabs.
- Prevent double-booking of a doctor's slot, and keep a live count of today's appointments in each doctor's entry.
- Tables must sit inside scroll panes and be updated through their models.

Constraints:

- No business logic may live in event-handler code beyond calls into the service layer.
- The three tabs must share a single set of repositories.
- Cancelling an appointment must free the slot and update every view that displays it.

Problem 64: Organization Chart Navigator

Scenario: A company's HR team wants to browse and maintain its organization structure (company, divisions, departments, and teams) with employees in each unit.

Requirements:

- Display the organization hierarchy in a `JTree`, within a scroll pane, built from your own model classes.
- Selecting a node displays its details (head, head-count, and total payroll, aggregated across child units) and its employees in a `JTable`.
- Provide actions to add a unit under the selected node, rename it, move an employee between units, and remove an empty unit, and update the tree and the details immediately.
- Respond to tree selection and expansion events through the appropriate listeners, and show the path of the current selection in a label.
- Display icons that differ for different node types.

Constraints:

- A unit containing employees or sub-units cannot be removed; the user is told why.
- Unit names must be unique among siblings.
- The aggregation of head-count and payroll must be correct after any change to the structure.

Problem 65: Loan EMI Calculator as an Applet

Scenario: A bank's older intranet pages embed a loan EMI calculator as an applet, and the bank needs the component rebuilt in a form that the syllabus calls `JApplet`, while the engineering team also wants it to survive in a modern desktop environment.

Requirements:

- Build the calculator as a `JApplet` with fields for principal, rate, and tenure, a loan-type combo box, a Calculate button, and a results area showing the EMI, total interest, and a yearly summary table.
- Use the applet life-cycle methods (initialization, start, stop, and destroy) meaningfully, for example by validating parameters supplied on the hosting page and by releasing resources when the applet stops.
- Provide a standalone desktop frame that hosts the same calculator logic and user interface, so the application is usable even when applets cannot be run on the installed JDK.
- Separate the calculation classes from the user-interface classes so that both the applet and the frame use the same logic.
- Include a short note, as comments or in a text file, that explains the applet's current status in the Java platform and how it affects design choices.

Constraints:

- Invalid inputs must be reported in the interface and never crash it.
- Results from the applet and the frame must be identical for identical input.
- Calculation code must not contain any user-interface references.

Problem 66: Retail Point-of-Sale Terminal

Scenario: A supermarket needs a desktop checkout terminal that scans products, manages a basket, takes payment, and prints a receipt.

Requirements:

- Provide a product code field and a product combo box, a basket table with quantity editing and line removal, running totals, and check boxes for options such as carry-bag purchase and loyalty-card use.
- Offer radio buttons for payment mode (cash, card, wallet) with mode-specific fields and rules (for example, change calculation for cash).
- When Pay is pressed, perform payment authorization in a background thread so the interface stays responsive, update the Swing components safely from that thread, and show the progress.
- Reduce stock when a sale completes, and refuse items that are out of stock.
- Build the receipt text with `StringBuffer` and show it in a scrollable text area, and keep a shift summary of sales by payment mode.

Constraints:

- A sale is committed once: pressing Pay repeatedly must not charge twice.
- Inventory updates must be thread-safe if several terminals share the same stock.
- Failed authorization must leave the basket and stock intact and offer a retry.

Problem 67: Live Train Status Board

Scenario: A railway station's control room wants a live status display that shows arriving and departing trains, updated automatically as delay reports come in from several sources.

Requirements:

- Display trains in a table within a scroll pane with columns for train number, name, platform, scheduled and expected times, delay, and status, with colors or icons for on time, delayed, and cancelled.
- Run several report-feeder threads that submit delay updates, platform changes, and cancellations to a shared status model at random intervals.

- Update the display as changes occur, without blocking the interface and without modifying the Swing components from a non-event thread.
- Provide a combo box filter (all, arrivals, departures, delayed only) and a text field search, and a pause and resume control that suspends the display updates, but not the incoming reports.
- Provide a log tab, in a tabbed pane, that lists every update received, and an end-of-run summary of reports processed by each feeder.

Constraints:

- Concurrent updates to the shared model must not be lost or applied out of order for the same train.
- Pausing the display must not lose updates; they must appear on resume.
- The program must shut down cleanly, stopping all feeder threads when the window closes.

Problem 68: Multi-Counter Cinema Booking Workstation

Scenario: A multiplex wants a training application in which a supervisor watches several simulated booking counters compete for the same seats while one real operator also books seats manually.

Requirements:

- Show the seat map as a grid of buttons whose color and icon reflect available, held, and sold seats, and update it in real time.
- The operator selects seats with the mouse and confirms the booking with a button; a combo box selects the show, and radio buttons select the ticket category.
- Run several simulated counter threads that book random seats concurrently with the operator, and show each counter's activity in a scrollable log and a table of counts.
- Seats must be held briefly during selection and released if the booking is not confirmed within a time limit, using a background timer thread.
- Detect and refuse conflicting requests, informing the operator immediately when a seat that they selected has been taken.

Constraints:

- No seat may ever be sold or held twice; verify the final state by code.
- Seat button updates must occur safely on the event thread.
- Closing the window must stop every counter thread and the timer thread.

Problem 69: Insurance Policy Administration Workbench

Scenario: An insurance company's service staff want a single desktop workbench in which they locate customers, review their policies and claims, and process changes.

Requirements:

- Show customers and their policies in a `JTree`, with a details area to the right in a tabbed pane containing an editable policy form (text fields, combo boxes, radio buttons, and check boxes for riders), a claims table inside a scroll pane, and a premium and payment history view.
- Build on a class hierarchy for policy types, an interface for premium calculation, and a generic repository.
- Support registering a new policy, adding a rider, filing a claim, and moving a claim through its workflow stages, using exceptions for rule violations and displaying them as precise messages.
- Premium figures update live as the user changes form fields, and the tree reflects every change.
- Provide a search box that filters the tree by customer name or policy number.
- Place the model, services, and user interface in separate packages.

Constraints:

- Users may only edit fields that the policy's current state permits.
- Unsaved changes must be flagged, and switching to another tree node must prompt the user to save or discard them.
- A claim must follow the workflow order and cannot exceed the policy limit.

Problem 70: Integrated College Administration Suite

Scenario: A college wants a single desktop suite that combines student records, course offerings, faculty assignments, timetable management, and fee processing, replacing its separate legacy tools.

Requirements:

- Design and implement the suite in several packages (model, repositories, services, concurrency or background tasks, user interface), with a class hierarchy for people (students, faculty, staff), interfaces for fee and eligibility rules, and generic repositories with a common contract.
- Include modules for student admission and profile management, course registration with prerequisite and seat rules, faculty workload assignment with timetable clash detection, and fee collection with instalments, fines, and receipts.

- Provide a navigation tree (departments, programs, and batches), a tabbed workspace for each module, tables with scroll panes for lists, and forms using text fields, combo boxes, radio buttons, and check boxes.
- Run background jobs in separate threads: a fee-reminder generator, a nightly consolidation of attendance and fees, and a report builder that produces text summaries using `StringBuffer`, with coordination between jobs that depend on one another.
- Define a user-defined exception hierarchy covering validation, rule, and concurrency-related failures, and present each failure type appropriately.
- Accept command-line arguments to choose a startup mode (empty database, demo data of a requested size) and report unrecognized arguments clearly.
- Include a final test scenario that demonstrates the correctness of the main rules, including a concurrent registration for the last seat in a course.

Constraints:

- Business rules must be independent of the user interface and testable without it.
- Shared data must remain consistent under concurrent access, and background jobs must not freeze the interface.
- The application must start, run, and shut down cleanly with no stray threads.
- Provide build and run instructions that use packages and an explicit classpath.

Syllabus Coverage Checklist

This checklist maps each prescribed topic to the problems that give it meaningful practice. It is a reference for the student, and the problems themselves are not grouped this way.

Unit I: Fundamentals, Classes and Objects

- Java history and buzzwords: 2 (see supplementary study note below).
- Data types, variables, arrays, operators, control statements: 1, 3, 4, 11, 12, 14.
- Classes, objects, methods, constructors, types of constructors, and constructor overloading: 5, 7, 13.
- Static members and static initialization: 6, 10, 13.
- Access control: 13, 21, 22, 39.
- The `this` keyword: 5, 7, 13.
- Garbage collection and `finalize()`: 10.
- Method overloading: 3, 4, 5, 7, 11, 12, 14.
- Parameter-passing mechanisms: 8.

- The `final` keyword: 6, 8, 16, 21.
- Nested and inner classes: 9, 27.

Unit II: Inheritance, Interfaces, Packages and Strings

- Inheritance, access specifiers, `super`, and overriding: 15, 16, 17, 21.
- `final` with inheritance: 16, 17, 21.
- Abstract classes and dynamic method dispatch: 15, 16, 17, 19, 26, 28, 42.
- The `Object` class: 20, 26, 28, 34.
- Interfaces (definition, implementation, variables, and interface inheritance): 18, 19, 26, 27, 28, 41, 44.
- Packages, classpath, access protection, and importing: 22, 39, 44, 57, 69, 70.
- `String` and `StringBuffer`: 4, 23, 24, 25, 39, 40, 41, 51, 61, 66, 70.
- Command-line arguments: 4, 25, 70.

Unit III: Exceptions, Multithreading, Generics and Collections

- Exception handling (`try`, `catch`, `throw`, `throws`, `finally`, multiple catch, nested try, and built-in exceptions): 29, 31, 32.
- User-defined exceptions: 30, 31, 32, 36, 38, 41, 42, 43, 44, 51, 54, 57, 58, 63, 69, 70.
- Thread model, life cycle, `Thread`, `Runnable`, and multiple threads: 45, 46, 47, 50, 51.
- Synchronization: 45, 46, 47, 54, 55, 57, 58.
- Inter-thread communication: 48, 52, 53, 55, 56, 58.
- Deadlock: 47, 49, 53, 57.
- Generic classes and methods: 36, 37, 38, 52, 56, 57, 63, 69, 70.
- Collection interfaces, collection classes, and iterators: 33, 34, 35, 38, 40, 41, 43, 44.

Unit IV: Event Handling and Swing

- Events, event sources, event classes, listeners, and the delegation event model: 59 through 70.
- `JFrame` and `JComponent`: 59, 62.
- Icons, labels, text fields, and `JButton`: 59.
- Check boxes, radio buttons, and combo boxes: 60.
- Scroll panes and tables: 61, 63, 67.
- Tabbed panes: 63, 67, 69, 70.
- Trees: 64, 69, 70.
- `JApplet`: 65.

Topics That Need Supplementary Study

Some syllabus items are conceptual or tied to older parts of the platform, so a standalone program cannot teach them fully. Please read about the following in your textbook alongside the problems.

- **Java history and buzzwords:** Problem 2 gives them a practical anchor, but the reasoning behind each design goal is best studied from the text.
- **`finalize()`:** Problem 10 makes you observe it, but it is deprecated and its behavior is not guaranteed. Study why explicit resource release is the recommended approach in modern code.
- **`JApplet`:** Problem 65 covers it as the syllabus requires, but applet support has been removed or deprecated in current JDKs, so you may need the desktop host frame described there to run it.
- **AWT:** The course outcomes mention AWT and Swing, but the unit content lists only Swing, so this set follows the unit content. Swing builds on AWT events and components, so a brief read of AWT basics will help.
- **Garbage collection internals, thread scheduling, and priorities:** Programs can only observe their effects, which vary between JVMs and runs. Read the theory to interpret what you see.
- **Supporting classes:** A few problems use ordinary standard-library classes that are not listed in the syllabus (such as `Scanner`, `Math`, and `SwingUtilities` for safe interface updates). Treat them as supporting tools rather than syllabus topics.